

Product Research Assistant

System Architecture, Capabilities, and Deployment Guide

1. System Overview

The Product Research Assistant is a full-stack, AI-powered agentic system designed to help users search, compare, and select consumer electronics (Laptops, Tablets, Smartphones, Audio Devices, and Monitors) under any budget constraints. The system leverages the Model Context Protocol (MCP) to access database tools and provides a responsive split-screen React dashboard for interactive chat and product comparison.

2. Architecture & Components

The system is divided into four main layers:

- **React Frontend:** A Tailwind CSS dashboard providing a split-screen user interface. The left panel serves a chat conversation with the agent, while the right panel dynamically populates search cards, a side-by-side comparison matrix, and details of the recommended overall best product.
- **FastAPI Backend:** A Python web application that compiles and serves the React static assets, implements stateless session management, and exposes the `/run` and `/run_sse` agent runtimes.
- **Google ADK Agent:** Orchestrates execution flow using the lightweight, fast **gemini-3.5-flash-lite** model, coordinating tools to retrieve and formulate recommendations.
- **MCP Server:** A standalone service using FastMCP transport to host tools: `search_products`, `get_product_details`, and `compare_products`, connected directly to a catalog database containing 26 electronics items with verified manufacture hyperlinks.

3. Core Capabilities & Phase 2 Upgrades

To scale the prototype into a fully featured electronic research assistant, the following key upgrades were made:

3.1. Multi-Category Expansion

The dataset was expanded from simple laptops to five distinct electronics categories (Laptops, Tablets, Smartphones, Headphones/Audio, Monitors). Additionally, the keyword processing inside the search tool was upgraded to map user search queries like 'tab' or 'screen' directly to categories like 'Tablet' or 'Monitor'.

3.2. Latency Fix (Gemini 3.5 Flash Lite)

To fix the 500/504 gateway timeout issues caused by deprecation and high demand of legacy models, the agent model was switched to **gemini-3.5-flash-lite**. This resulted in a massive speed increase, dropping the request latency from 200+ seconds to less than 15 seconds.

3.3. Clickable Product Hyperlinks

Verified product URLs were added for all 26 catalog items. The React frontend was upgraded to display these URLs as clickable hyperlinks using the *ExternalLink* icon across search results cards, best choice recommendations, and comparative tables.

4. Local Development Setup

Follow these steps to run the complete stack locally:

Step 1: Start the MCP Server

Activate the backend virtual environment, navigate to the MCP server directory, and boot the server on port 8081:

```
cd mcp_server ..\backend\.venv\Scripts\python server.py 8081
```

Step 2: Start the FastAPI Backend

In a separate terminal, navigate to the backend directory and start the server on port 8080:

```
cd backend .env file setup: GEMINI_API_KEY=your_gemini_key  
PRODUCT_MCP_URL=http://localhost:8081/sse PORT=8080 .venv\Scripts\python main.py
```

5. Cloud Run Deployment

Both backend and MCP services are packaged as containerized Docker applications and deployed to Google Cloud Run:

- **Build MCP Server Container:** *gcloud builds submit --tag us-central1-docker.pkg.dev/[PROJECT_ID]/mcp-repo/product-mcp-server*
- **Deploy MCP Server:** *gcloud run deploy product-mcp-server --image ... --port 8081*
- **Build Backend Container:** *gcloud builds submit --tag us-central1-docker.pkg.dev/[PROJECT_ID]/adk-repo/product-research-backend*
- **Deploy Backend:** *gcloud run deploy product-research-backend --image ... --set-env-vars PRODUCT_MCP_URL=https://[MCP_RUN_URL]/sse,GEMINI_API_KEY=... --port 8080*